PPWL

Cheatsheet, Latihan, dan Kunci Jawaban

Persiapan quiz live coding Laravel, PHP native, CRUD, migration, seeder, Blade, controller, dan query.

Cheatsheet quiz live coding PPWL

Disusun dari modul Bab I–II, laporan pertemuan 2–6, dan kode proyek Buku/Event dalam folder PPWL. Folder pertemuan 1 tidak ditemukan; dasar PHP dari Bab I tetap dimasukkan. Ini bahan belajar, bukan prediksi soal. Instruksi tugas dalam laporan dibaca sebagai materi, bukan perintah untuk mengubah proyekmu.

1. Hafalan pertama: kenali kata kerja soal

Jika diminta…	Jawaban inti
Cek PHP	`php -v`
Jalankan file PHP di terminal	`php latihan.php`
Jalankan web PHP native	`php -S localhost:8000`
Jalankan Laravel	`php artisan serve`
Buat controller	`php artisan make:controller BukuController`
Buat model	`php artisan make:model Buku`
Buat model sekaligus migration	`php artisan make:model Buku -m`
Buat file migration	`php artisan make:migration create_books_table --create=books`
Jalankan migration	`php artisan migrate`
Buat file seeder	`php artisan make:seeder BukuSeeder`
Jalankan seeder tertentu	`php artisan db:seed --class=BukuSeeder`
Jalankan DatabaseSeeder	`php artisan db:seed`
Cek route	`php artisan route:list`

Pola: `make:` membuat file; `migrate` menerapkan struktur; `db:seed` mengisi data. Membuat file belum menjalankan isinya.

2. CMD, PHP, Composer, dan instalasi

Contoh berikut untuk **Windows CMD**. Ganti lokasi sesuai proyek. Semua perintah Artisan dijalankan di folder yang berisi file `artisan`.

```
cd /d "D:\A CURRENT TASK\SEMESTER 3\PPWL\Pertemuan 4\laravel"
dir
php -v
where php
composer --version
php artisan --version
php artisan serve
```

`cd /d` pindah drive sekaligus folder; tanda petik diperlukan untuk lokasi dengan spasi. Di PowerShell cukup `cd "D:\lokasi proyek"`; pencarian executable memakai `where.exe php`.

Kebutuhan	Command / tindakan
PHP native lewat browser, dari folder file PHP	`php -S localhost:8000`, buka `http://localhost:8000/index.php`
PHP native lewat XAMPP	Start Apache; taruh proyek di `htdocs`; buka `http://localhost/nama-folder/`
Menjalankan skrip PHP di CMD	`php latihan.php` — output muncul di terminal
Cek sintaks PHP	`php -l latihan.php`
Laravel port lain	`php artisan serve --port=8001`
Hentikan server	`Ctrl+C` di terminal server
Proyek Laravel baru	`composer create-project laravel/laravel quiz-app`
Pilih versi Laravel, contoh 11	`composer create-project laravel/laravel quiz-app "^11.0"`
Pasang Laravel installer	`composer global require laravel/installer`
Buat proyek lewat installer	`laravel new quiz-app`

Kebutuhan	Command / tindakan
Pasang dependensi proyek yang sudah ada	`composer install`
Buat `.env` jika belum ada, CMD	`copy .env.example .env`
Buat application key	`php artisan key:generate`
Lihat bantuan command	`php artisan help make:seeder`

`composer install` memakai versi di `composer.lock` jika tersedia. `composer update` mencari versi yang memenuhi constraint dan memperbarui lock; jangan tertukar dengan membuat proyek baru.

Server PHP/Artisan tidak otomatis menyalakan MySQL. Jika proyek menggunakan MySQL lokal, layanan database tetap harus berjalan.

3. Peta file: soal kode harus dikerjakan di mana?

Bagian	Lokasi	Fungsi
Route web	`routes/web.php`	URL → handler/controller
Controller	`app/Http/Controllers/BukuController.php`	Mengolah request/data, mengirim view
Model	`app/Models/Buku.php`	Mewakili tabel dan operasi data
View	`resources/views/buku.blade.php`	Menampilkan data
Migration	`database/migrations/...create_books_table.php`	Struktur tabel
Seeder	`database/seeders/BukuSeeder.php`	Data awal
Pengaturan database	`.env`	Koneksi DB
CSS/JS biasa	`public/css/style.css`, `public/js/script.js`	Aset yang diakses browser

Alur: **browser → route → controller → model/database → controller → view → browser**. Route sederhana juga boleh langsung mengembalikan view.

4. Route, controller, dan mengirim data

Di `routes/web.php`:

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\BukuController;

Route::get('/about', function () {
    return view('about', ['name' => 'Reyhan']);
});

Route::get('/buku', [BukuController::class, 'index'])->name('buku.index');

Route::get('/halo/{nama}', function ($nama) {
    return view('halo', compact('nama'));
});
```

`view('about')` mencari `resources/views/about.blade.php`. `view('buku.index')` mencari `resources/views/buku/index.blade.php`. Nama URL dan nama view boleh berbeda.

Contoh controller lengkap:

```
<?php
namespace App\Http\Controllers;

use App\Models\Buku;

class BukuController extends Controller
{
    public function index()
    {
        $data_buku = Buku::all();
        return view('buku', compact('data_buku'));
    }
}
```

`compact('data_buku')` setara dengan `['data_buku' => $data_buku]`. View mengakses `$data_buku`; nama variabel harus cocok.

5. Blade: output, perulangan, kondisi, layout

```
<h1>{{ $name }}</h1>

@if ($total_buku > 0)
    <p>Ada buku.</p>
@else
    <p>Belum ada buku.</p>
@endif

@forelse ($data_buku as $buku)
    <p>{{ $loop->iteration }}. {{ $buku->judul }}</p>
@empty
    <p>Buku tidak ditemukan.</p>
@endforelse

<p>Rp {{ number_format($buku->harga ?? 0, 0, ',', '.') }}</p>
<link rel="stylesheet" href="{{ asset('css/style.css') }}">
<script src="{{ asset('js/script.js') }}"></script>
```

`@foreach ... @endforeach` untuk loop biasa; `@forelse ... @empty ... @endforelse` sekaligus menangani hasil kosong. `{{ }}` melakukan escaping HTML; `{!! !!}` mencetak HTML mentah, gunakan hanya untuk konten yang aman.

Template induk `resources/views/layouts/app.blade.php`:

```
<!doctype html>
<html><body>
    @include('partials.header')
    @yield('content')
</body></html>
```

Halaman anak `resources/views/about.blade.php`:

```
@extends('layouts.app')
@section('content')
    <h1>About</h1>
@endsection
```

`extends` memilih induk, `yield` menyiapkan tempat, `section` mengisi tempat, `include` menyertakan potongan view. Contoh induk memerlukan file `resources/views/partials/header.blade.php`.

6. Database dan migration

Contoh `.env` untuk database lokal yang sudah dibuat:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=toko_buku
DB_USERNAME=root
DB_PASSWORD=
```

Gunakan username/password sebenarnya. Jika perubahan `.env` belum terbaca: `php artisan config:clear`.

Di method `up()` file migration, dengan import `Schema` dan `Blueprint` bawaan:

```
Schema::create('books', function (Blueprint $table) {
    $table->id();
    $table->string('judul');
    $table->string('penulis');
    $table->integer('harga');
    $table->date('tgl_terbit');
    $table->timestamps();
});
```

Di method `down()`:

```
Schema::dropIfExists('books');
```

`up()` menerapkan perubahan; `down()` membatalkannya. `timestamps()` menambah `created_at` dan `updated_at`.

Perintah	Makna
`php artisan migrate`	Jalankan migration yang belum diterapkan
`php artisan migrate:status`	Lihat status migration
`php artisan migrate:rollback`	Batalkan batch migration terakhir
`php artisan migrate:rollback --step=1`	Batalkan satu migration terakhir

Perintah	Makna
`php artisan migrate --seed`	Jalankan migration lalu DatabaseSeeder
`php artisan migrate:fresh --seed`	**Hapus semua tabel pada koneksi terkait**, buat ulang, lalu seed
`php artisan migrate:refresh`	Rollback seluruh migration lalu jalankan lagi; data dapat hilang

Mengedit migration yang sudah dijalankan tidak membuat `migrate` menjalankannya ulang. Untuk menambah kolom pada tabel yang sudah ada, buat migration baru, misalnya `php artisan make:migration add_stok_to_books_table --table=books`, kemudian gunakan `Schema::table(...)`.

7. Model dan seeder — sesuai proyekmu

Model Buku dalam laporan menggunakan **tabel `books`**, bukan `buku`. CRUD native di Bab I menggunakan tabel **`buku`**. Cocokkan nama tabel dengan konteks soal.

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Buku extends Model
{
    protected $table = 'books';
    public $timestamps = false;
    protected $fillable = ['judul', 'penulis', 'harga', 'tgl_terbit'];
}
```

`$table` menentukan tabel. `$fillable` mengizinkan atribut pada mass assignment biasa seperti `Buku::create(...)`. `$timestamps = false` mematikan pengisian otomatis timestamp; tidak menghapus kolom timestamp. Dalam kode laporanmu, migration tetap mempunyai kolom timestamp tetapi model mematikan pengisiannya. Saat Laravel menjalankan seeder, proteksi mass assignment otomatis dinonaktifkan; `$fillable` tetap penting di penggunaan model di luar seeder.

Seeder lengkap, dibuat melalui `make:seeder`:

```
<?php
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Buku;

class BukuSeeder extends Seeder
{
    public function run(): void
    {
        Buku::create([
            'judul' => 'Belajar Laravel',
            'penulis' => 'Budi Santoso',
            'harga' => 75000,
            'tgl_terbit' => '2026-09-01',
        ]);
    }
}
```

Di method `run()` milik `DatabaseSeeder`, untuk menghubungkan seeder:

```
$this->call([
    BukuSeeder::class,
    EventSeeder::class,
]);
```

`db:seed --class=BukuSeeder` bisa menjalankan BukuSeeder langsung tanpa mendaftarkannya di DatabaseSeeder. Seeder berisi `create()` dapat menambah data duplikat jika dijalankan berulang.

8. Query, filter, search, dan statistik

Semua contoh `Buku::...` membutuhkan `use App\Models\Buku;` pada controller.

Soal	Potongan jawaban
Semua buku	`Buku::all()`
Buku id 3	`Buku::find(3)`
Buku penulis tertentu	`Buku::where('penulis', 'Budi Santoso')->get()`
Harga minimal 50000	`Buku::where('harga', '>=', 50000)->get()`
Harga paling mahal dahulu	`Buku::orderBy('harga', 'desc')->get()`

Soal	Potongan jawaban
Lima buku terbaru menurut id, sesuai laporan	`Buku::orderBy('id', 'desc')->take(5)->get()`
Lima terbaru menurut tanggal terbit	`Buku::orderBy('tgl_terbit', 'desc')->take(5)->get()`
Daftar penulis tanpa duplikat	`Buku::select('penulis')->distinct()->get()`
Jumlah seluruh buku	`Buku::count()`
Total harga seluruh buku	`Buku::sum('harga')`
Harga tertinggi/terendah seluruh buku	`Buku::max('harga')` / `Buku::min('harga')`

Cara pencarian persis pola laporan, di method `index(Request $request)`; tambahkan import `use Illuminate\Http\Request;`:

```
$nama_penulis = $request->penulis;
$keyword = $request->keyword;
$data_buku = Buku::all();

if ($nama_penulis && $nama_penulis != 'Semua Penulis') {
    $data_buku = Buku::where('penulis', $nama_penulis)->get();
}
if ($keyword) {
    $data_buku = $data_buku->filter(function ($buku) use ($keyword) {
        return stripos($buku->judul, $keyword) !== false;
    });
}

$total_buku = $data_buku->count();
$total_harga = $data_buku->sum('harga');
$harga_tertinggi = $data_buku->max('harga') ?? 0;
$harga_terendah = $data_buku->min('harga') ?? 0;
```

`stripos` mencari tanpa membedakan huruf besar/kecil. Hasil `0` berarti ketemu di awal teks, jadi harus dibandingkan dengan `!== false`. `use ($keyword)` membawa variabel ke closure.

Alternatif pencarian di database, jika soal tidak mewajibkan Collection:

```
$data_buku = Buku::where('judul', 'like', '%' . $keyword . '%')->get();
```

`%kata%` berarti mengandung kata. Perilaku huruf besar/kecil `LIKE` bergantung pada database/collation; jangan anggap identik pada semua database.

Form pencarian di Blade, dengan route bernama `buku.index` dan variabel `$keyword` sudah dikirim:

```
<form action="{{ route('buku.index') }}" method="GET">
    <input name="keyword" value="{{ $keyword }}">
    <button type="submit">Cari</button>
</form>
```

`name="keyword"` menentukan nama parameter yang dibaca `$request->keyword`. GET menaruh parameter di URL. Form POST pada route web umumnya memerlukan `@csrf`.

Detail Event dalam laporan:

```
use App\Models\Event;

$event_terbaru = Event::orderBy('tanggal_mulai', 'desc')->take(5)->get();
$events = Event::where('tipe_event', 'Trail Run')->get();
$total_event = Event::count();
$total_trail_run = Event::where('tipe_event', 'Trail Run')->count();
```

Statistik Buku berasal dari `$data_buku` yang telah difilter; statistik Event di laporan berasal dari query seluruh tabel. `get()`/`all()` menghasilkan Collection berisi banyak model; `first()`/`find()` menghasilkan satu model atau `null`. `where()` sebelum `get()` adalah query database; `filter()` setelah `get()`/`all()` bekerja pada Collection di memori.

9. PHP dasar dan CRUD native

```
<?php
$nama = 'Reyhan';
$harga = 75000;
echo 'Halo ' . $nama; // titik = penggabung string
echo 10 + 5;

function tambah($a, $b = 0) {
    return $a + $b;
}
echo tambah(2, 3);

echo strlen('Hello world');          // 11, panjang dalam byte
```

```
echo str_word_count('Hello world');  // 2
echo strrev('abc');                  // cba
echo strpos('Hello world', 'world'); // 6, dimulai dari indeks 0
echo str_replace('world', 'Reyhan', 'Hello world');
```

Variabel `$nama` dan `$Nama` berbeda. String petik ganda dapat menginterpolasi variabel; petik tunggal tidak. Komentar: `//`, `#`, atau `/* ... */`. `$_GET['id']` dari query URL; `$_POST['judul']` dari form POST.

Contoh koneksi dan pembacaan native:

```
$koneksi = mysqli_connect('localhost', 'root', '', 'toko_buku');
if (!$koneksi) {
    die('Koneksi gagal: ' . mysqli_connect_error());
}
$hasil = mysqli_query($koneksi, 'SELECT * FROM buku');
while ($buku = mysqli_fetch_assoc($hasil)) {
    echo htmlspecialchars($buku['judul'], ENT_QUOTES, 'UTF-8');
}
```

Parameter koneksi: host, username, password, nama database. `mysqli_fetch_assoc()` mengambil satu baris sebagai array dengan key nama kolom. Native: `$buku['judul']`; Eloquent: `$buku->judul`.

SQL berikut dijalankan di database/SQL client, **bukan langsung diketik sebagai command CMD**:

```
CREATE DATABASE toko_buku;
INSERT INTO buku (judul, penulis, harga, tgl_terbit)
VALUES ('Bumi Manusia', 'Pramoedya Ananta Toer', 95000, '1980-08-10');
SELECT * FROM buku ORDER BY harga DESC;
UPDATE buku SET harga = 120000 WHERE id = 3;
DELETE FROM buku WHERE id = 7;
```

Setelah proses simpan, redirect native: `header('Location: index.php'); exit;` sebelum output HTML. Hidden input `id` membawa identitas baris saat edit. Input pengguna untuk SQL gunakan prepared statement; `UPDATE`/`DELETE` tanpa `WHERE` dapat mengenai seluruh baris.

10. Kalau mendadak error

Gejala	Cek / tindakan pertama
`Could not open input file: artisan`	Salah folder; masuk folder yang memiliki `artisan`
`php is not recognized`	Cek PATH PHP; `where php`, atau panggil executable dengan path lengkap
Port 8000 terpakai	`php artisan serve --port=8001`
`View [buku] not found`	Cek `resources/views/buku.blade.php` dan nama pada `view()`
`Undefined variable $data_buku`	Kirim data dengan `compact('data_buku')` / array
`Class ... not found`	Cek nama class, namespace, dan `use`
`Table ... doesn't exist`	Cek database, nama tabel model, dan status migration
DB connection refused	Cek MySQL berjalan, host, dan port
Seeder baru tidak berjalan lewat `db:seed`	Daftarkan di DatabaseSeeder atau pakai `--class=...`
Kolom `created_at` tidak ada	Cocokkan timestamps model dengan struktur tabel
Perubahan `.env` tidak terbaca	`php artisan config:clear`
Route lama masih muncul karena cache	`php artisan route:clear`
View cache bermasalah	`php artisan view:clear`

11. Prioritas belajar malam ini, 90 menit

1. **15 menit:** hafalkan tabel command pertama; tulis ulang tanpa melihat.
2. **20 menit:** route → controller → view; hafalkan lokasi file dan `compact()`.
3. **20 menit:** bedakan migration/model/seeder dan command membuat vs menjalankan.
4. **20 menit:** query, filter, search, statistik Buku/Event.
5. **15 menit:** ambil soal acak dari LATIHAN.md; jawab maksimal 60 detik, cek kunci setelah menjawab.

Kalau waktu tinggal 30 menit: 10 menit command, 10 menit route/Blade, 10 menit seeder/query. Ini urutan belajar berdasarkan cakupan praktik, bukan klaim probabilitas soal.

12. Catatan sumber dan versi

Proyek pertemuan 4 mensyaratkan `laravel/framework: ^11.31` dalam `composer.json`. Fokuskan sintaks latihan pada proyek yang dipakai saat quiz, cek dengan `php artisan --version`.

Tabel fitur versi dalam laporan pertemuan 3 jangan dihafal mentah sebagai fakta resmi: misalnya Concurrency dan deferred functions sudah diperkenalkan dalam siklus Laravel 11. Jangan menganggap setiap proyek punya empat file route; struktur berbeda antarversi/paket. `web.php` tetap lokasi utama latihan route web ini. Ini koreksi materi, bukan perubahan terhadap laporan.

Sumber lokal yang dibaca:

• Modul: `PPWL/Pertemuan 2/BAB I - Kilas Balik CRUD PHP dan Pengenalan Laravel (update).pdf`.

• Laporan 2: `PPWL/Pertemuan 2/PPW2_A1_PERTEMUAN 2_562000_REYHAN ABELARD FIKRI_rev2.pdf`.

• Laporan 3: `PPWL/Pertemuan 3/PPW2_A1_PERTEMUAN 3_562000_REYHAN ABELARD FIKRI.pdf`.

• Modul: `PPWL/Pertemuan 4/BAB II.pdf`.

• Laporan 4: `PPWL/Pertemuan 4/final/PPW2_A1_PERTEMUAN 4_562000_REYHAN ABELARD FIKRI.pdf`.

• Laporan 5: `PPWL/Pertemuan 5/PPW2_A1_PERTEMUAN 5_562000_REYHAN ABELARD FIKRI.pdf`.

• Laporan 6: `PPWL/Petemuan 6/PPW2_A1_PERTEMUAN 6_562000_REYHAN ABELARD FIKRI.pdf`, serta kode di `Latex/Kode`.

Verifikasi resmi: [PHP built-in server](https://www.php.net/manual/en/features.commandline.webserver.php), [Laravel seeding](https://laravel.com/framework/docs/12.x/seeding), [migration](https://laravel.com/framework/docs/12.x/migrations), [routing](https://laravel.com/framework/docs/11.x/routing), [Blade](https://laravel.com/framework/docs/11.x/blade), [Eloquent](https://laravel.com/framework/docs/11.x/eloquent), [release Laravel 12](https://laravel.com/framework/docs/12.x/releases).

Latihan acak satu langkah + studi kasus PPWL

Cara latihan

Pilih angka 1–45 secara acak. Baca hanya satu soal, jawab dengan command atau potongan kode, lalu cek nomor yang sama di KUNCI.md. Target 30–60 detik. Latihan command database yang mengubah data gunakan database latihan sendiri. Tidak perlu membuat seluruh aplikasi jika soal hanya meminta satu langkah.

Untuk soal Artisan, anggap terminal sudah berada di root proyek dan PHP tersedia, kecuali soal menyebut lain. Untuk soal query, model dan tabel sudah tersedia. Import yang diperlukan tetap menjadi bagian jawaban jika belum ada.

A. CMD, PHP, dan Composer

1. CMD sedang berada di drive C. Pindah ke folder `D:\A CURRENT TASK\SEMESTER 3\PPWL\Pertemuan 4\laravel` dalam satu command.

2. Tampilkan versi PHP yang sedang dipakai terminal.

3. Jalankan file `latihan.php` agar hasilnya tampil di terminal.

4. Kamu berada di folder berisi `index.php` native. Nyalakan server di localhost port 8000 agar file dapat dibuka di browser.

5. Kamu berada di root Laravel. Jalankan development server.

6. Port 8000 dipakai. Jalankan Laravel di port 8001.

7. Buat proyek Laravel baru bernama `quiz-app` melalui Composer.

8. Buat proyek `quiz11` dengan constraint Laravel 11 melalui Composer.

9. Proyek sudah diunduh, `composer.lock` tersedia, folder vendor belum ada. Pasang dependensinya.

10. Tampilkan versi Laravel proyek aktif.

B. Membuat file vs menjalankan proses

11. Buat controller bernama `PostController`.

12. Buat model bernama `Buku` sekaligus migration-nya.

13. Buat migration untuk membuat tabel `books` dengan opsi nama tabel eksplisit.

14. Jalankan semua migration yang belum diterapkan.

15. Lihat status migration.

16. Batalkan satu migration terakhir.

17. Buat file seeder bernama `EventSeeder`.

18. Jalankan hanya `EventSeeder` tanpa melalui DatabaseSeeder.

19. Jalankan DatabaseSeeder.

20. Penguji secara eksplisit meminta seluruh tabel database latihan dibuang, dibangun ulang, lalu diisi lewat DatabaseSeeder. Tulis command.

C. Route, controller, dan Blade

21. Pada `routes/web.php`, buat route GET `/about` yang menampilkan view `about`.

22. Hubungkan GET `/posts` ke method `index` dari `PostController`; tulis import controller-nya juga.

23. Di controller, variabel `$data_buku` sudah tersedia. Kembalikan view `buku` sambil mengirim variabel itu.

24. Tulis lokasi file yang dicari oleh `view('buku.index')`.

25. Pada Blade, tampilkan `$name` dengan escaping HTML.

26. Tampilkan semua judul dari `$data_buku` memakai `@foreach`.

27. Tampilkan judul, tetapi jika `$data_buku` kosong tampilkan “Buku tidak ditemukan.”

28. Di layout induk, sediakan tempat untuk section `content`.

29. Di halaman anak, gunakan layout `layouts.app` dan isi section `content` dengan `<h1>About</h1>`.

30. Hubungkan file `public/css/style.css` dari Blade menggunakan helper aset.

D. Model, migration, seeder, dan query

31. Di model Buku, tentukan secara eksplisit bahwa tabelnya bernama `books`.

32. Di model, nonaktifkan pengisian timestamp otomatis.

33. Di dalam `Schema::create`, buat kolom `tgl_terbit` bertipe tanggal.

34. Pada method `run()` DatabaseSeeder, panggil BukuSeeder dan EventSeeder.

35. Di controller, ambil lima Buku dengan id terbesar terlebih dahulu.

36. Ambil Buku yang penulisnya `Budi Santoso`.

37. Ambil daftar penulis unik melalui model Buku.

38. `$data_buku` adalah Collection hasil filter. Hitung jumlah buku dan total harganya.

39. Ambil lima Event dengan `tanggal_mulai` paling besar terlebih dahulu.

40. Hitung jumlah semua Event bertipe `Trail Run` langsung di database.

E. Search, PHP native, dan diagnosis

41. Di controller yang menerima `Request $request`, ambil parameter query string `keyword` ke `$keyword`.

42. `$data_buku` adalah Collection dan `$keyword` sudah tersedia. Filter judul dengan `stripos` tanpa membedakan huruf besar/kecil.

43. Native PHP: tulis SQL untuk mengubah harga buku id 3 menjadi 120000.

44. Native PHP: `$buku` berasal dari `mysqli_fetch_assoc()`. Tampilkan judulnya dengan escaping HTML.

45. `php artisan serve` menghasilkan `Could not open input file: artisan`. Apa yang diperbaiki terlebih dahulu?

Studi kasus 1 — Toko Buku: pilih satu kartu

Konteks: model `App\Models\Buku` memakai tabel `books`, atribut `judul`, `penulis`, `harga`, `tgl_terbit`. Controller `BukuController`, view `buku.blade.php`. Masing-masing kartu adalah soal terpisah; tidak perlu mengerjakan seluruh alur untuk menjawab satu kartu.

B1. Buat controller BukuController lewat terminal.

B2. Buat route `/buku` menuju `index` dan beri nama `buku.index`.

B3. Tulis method `index()` yang mengambil semua Buku lalu mengirimnya sebagai `$data_buku` ke view `buku`.

B4. Pada method `run()` BukuSeeder, tambahkan satu buku: judul “Belajar Laravel”, penulis “Budi Santoso”, harga 75000, tanggal terbit 2026-09-01. Tulis import model yang diperlukan.

B5. Tulis form GET menuju route `buku.index` dengan input bernama `keyword`; pertahankan nilai `$keyword` setelah submit.

B6. Dalam method `index(Request $request)`, tulis logika filter penulis dari request. Nilai `Semua Penulis` berarti jangan filter.

B7. Pada daftar harga, tampilkan `75000` sebagai `Rp 75.000` menggunakan nilai `$buku->harga`.

B8. Pencarian tidak menghasilkan buku. Statistik diminta mengikuti hasil yang ditampilkan. Tulis jumlah dan total harga berdasarkan `$data_buku`, lalu sebutkan hasilnya jika kosong.

Studi kasus 2 — Trail Run: pilih satu kartu

Konteks: `App\Models\Event`, tabel `events`, atribut `nama_event`, `penyelenggara`, `tipe_event`, `tanggal_mulai`, `tanggal_selesai`. Tipe: `Run`, `Trail Run`, `Challenge`.

E1. Buat seeder EventSeeder lewat terminal.

E2. Jalankan hanya EventSeeder.

E3. Ambil semua event tipe Challenge sebagai `$events`.

E4. Ambil lima event terbaru berdasarkan `tanggal_mulai`.

E5. `$events` sudah difilter tipe. Terapkan pencarian nama menggunakan `$keyword` dengan pola Collection dari laporan.

E6. Tulis query jumlah semua event dan jumlah event Run. Statistik harus tetap menghitung seluruh database walaupun daftar difilter.

E7. Di Blade, tampilkan tanggal mulai sebagai `27-09-2026` jika nilainya `2026-09-27`.

E8. Mengapa pencarian tanpa hasil bisa menampilkan tabel kosong tetapi statistik Event tetap 20? Jelaskan sesuai kode laporan.

Studi kasus 3 — Penguji menyuruh memperbaiki satu bagian

D1. BukuSeeder sudah dibuat, tetapi `php artisan db:seed` hanya menjalankan seeder bawaan. Berikan satu perbaikan kode agar BukuSeeder ikut berjalan.

D2. Route `/buku` sudah benar, tetapi muncul `View [buku] not found`. File view yang dibuat ternyata `resources/views/books.blade.php`. Berikan salah satu perbaikan.

D3. View memakai `$data_buku`, tetapi controller hanya menulis `return view('buku');`. Perbaiki satu baris itu; anggap `$data_buku` sudah berisi Collection.

D4. Model Buku mencoba tabel yang namanya tidak sesuai. Tabel asli adalah `books`. Tulis satu properti model untuk memperbaikinya.

D5. Kode pencarian memakai `if (stripos($judul, $keyword))`. Pencarian gagal jika kata ditemukan di awal judul. Perbaiki kondisi.

D6. `.env` sudah diganti tetapi konfigurasi database masih memakai nilai lama karena cache. Tulis command yang sesuai.

D7. Migration baru selesai dibuat, tetapi tabel belum ada. Tulis command yang masih perlu dijalankan.

D8. Penguji menunjukkan `DELETE FROM buku;` untuk menghapus hanya id 7. Perbaiki SQL.

Penilaian mandiri

• **2 poin:** command/kode tepat, lokasi tepat, dapat menjelaskan fungsinya.

• **1 poin:** konsep benar tetapi ada salah nama, import, atau tanda baca.

• **0 poin:** tertukar fungsi command atau tidak dapat menjawab.

Catat soal yang nilainya 0–1, lalu ulang hanya soal tersebut setelah 15 menit. Kemampuan mengetik dari ingatan lebih relevan untuk live coding dibanding hanya merasa familier saat membaca.

Kunci latihan PPWL

Gunakan setelah menjawab LATIHAN.md. Potongan kode ditempatkan pada file/method yang disebut soal. Contoh `Buku::...` di controller membutuhkan `use App\Models\Buku;`, dan `Event::...` membutuhkan `use App\Models\Event;`.

Kunci soal 1–20

No.	Jawaban
1	`cd /d "D:\A CURRENT TASK\SEMESTER 3\PPWL\Pertemuan 4\laravel"`
2	`php -v`
3	`php latihan.php`
4	`php -S localhost:8000`, lalu buka `http://localhost:8000/index.php`
5	`php artisan serve`
6	`php artisan serve --port=8001`
7	`composer create-project laravel/laravel quiz-app`
8	`composer create-project laravel/laravel quiz11 "^11.0"`
9	`composer install`
10	`php artisan --version`
11	`php artisan make:controller PostController`
12	`php artisan make:model Buku -m`
13	`php artisan make:migration create_books_table --create=books`
14	`php artisan migrate`
15	`php artisan migrate:status`
16	`php artisan migrate:rollback --step=1`
17	`php artisan make:seeder EventSeeder`
18	`php artisan db:seed --class=EventSeeder`
19	`php artisan db:seed`
20	`php artisan migrate:fresh --seed` — menghapus semua tabel pada koneksi terkait

Kunci soal 21–30

21. Di `routes/web.php` (import Route biasanya sudah ada):

```
Route::get('/about', function () {
    return view('about');
});
```

22.

```
use App\Http\Controllers\PostController;
Route::get('/posts', [PostController::class, 'index']);
```

23. `return view('buku', compact('data_buku'));`

24. `resources/views/buku/index.blade.php`.

25. `{{ $name }}`.

26.

```
@foreach ($data_buku as $buku)
    <p>{{ $buku->judul }}</p>
@endforeach
```

27.

```
@forelse ($data_buku as $buku)
```

```
    <p>{{ $buku->judul }}</p>
@empty
    <p>Buku tidak ditemukan.</p>
@endforelse
```

28. `@yield('content')`.

29.

```
@extends('layouts.app')
@section('content')
    <h1>About</h1>
@endsection
```

30. `<link rel="stylesheet" href="{{ asset('css/style.css') }}">`.

Kunci soal 31–45

31. `protected $table = 'books';`

32. `public $timestamps = false;`

33. `$table->date('tgl_terbit');`

34.

```
$this->call([BukuSeeder::class, EventSeeder::class]);
```

35. `$buku_terbaru = Buku::orderBy('id', 'desc')->take(5)->get();`

36. `$data_buku = Buku::where('penulis', 'Budi Santoso')->get();`

37. `$penulis = Buku::select('penulis')->distinct()->get();`

38.

```
$total_buku = $data_buku->count();
$total_harga = $data_buku->sum('harga');
```

39. `$event_terbaru = Event::orderBy('tanggal_mulai', 'desc')->take(5)->get();`

40. `$total_trail_run = Event::where('tipe_event', 'Trail Run')->count();`

41. `$keyword = $request->keyword;` atau `$keyword = $request->input('keyword');`.

42.

```
$data_buku = $data_buku->filter(function ($buku) use ($keyword) {
    return stripos($buku->judul, $keyword) !== false;
});
```

43. `UPDATE buku SET harga = 120000 WHERE id = 3;`

44. `echo htmlspecialchars($buku['judul'], ENT_QUOTES, 'UTF-8');`

45. Pindah ke root proyek yang berisi `artisan`; di CMD cek dengan `dir`. Command benar tetapi direktori kerjanya salah.

Studi kasus Buku

B1. `php artisan make:controller BukuController`.

B2. Di `routes/web.php`:

```
use App\Http\Controllers\BukuController;
Route::get('/buku', [BukuController::class, 'index'])->name('buku.index');
```

B3. Pada BukuController dengan import Buku:

```
public function index()
{
    $data_buku = Buku::all();
    return view('buku', compact('data_buku'));
}
```

B4. Import berada di bagian atas file; panggilan `create` berada dalam `run()`:

```
use App\Models\Buku;

// Dalam run():
Buku::create([
    'judul' => 'Belajar Laravel',
    'penulis' => 'Budi Santoso',
    'harga' => 75000,
    'tgl_terbit' => '2026-09-01',
]);
```

B5.

```
<form action="{{ route('buku.index') }}" method="GET">
    <input type="text" name="keyword" value="{{ $keyword }}">
    <button type="submit">Cari</button>
</form>
```

B6. Dengan import Buku dan Request di controller:

```
$nama_penulis = $request->penulis;
$data_buku = Buku::all();
if ($nama_penulis && $nama_penulis != 'Semua Penulis') {
    $data_buku = Buku::where('penulis', $nama_penulis)->get();
}
```

B7. `Rp {{ number_format($buku->harga, 0, ',', '.') }}`.

B8.

```
$total_buku = $data_buku->count();
$total_harga = $data_buku->sum('harga');
```

Jika Collection kosong: jumlah 0, total harga 0. `Buku::count()` menghitung seluruh tabel sehingga tidak memenuhi permintaan statistik hasil filter ini.

Studi kasus Trail Run

E1. `php artisan make:seeder EventSeeder`.

E2. `php artisan db:seed --class=EventSeeder`.

E3. `$events = Event::where('tipe_event', 'Challenge')->get();`

E4. `$event_terbaru = Event::orderBy('tanggal_mulai', 'desc')->take(5)->get();`

E5.

```
if ($keyword) {
    $events = $events->filter(function ($event) use ($keyword) {
        return stripos($event->nama_event, $keyword) !== false;
    });
}
```

E6.

```
$total_event = Event::count();
$total_run = Event::where('tipe_event', 'Run')->count();
```

E7. `{{ date('d-m-Y', strtotime($event->tanggal_mulai)) }}`.

E8. Daftar diambil dari `$events` yang sudah difilter, tetapi statistik menggunakan `Event::count()` terhadap seluruh tabel. Angka 20 berlaku jika database masih berisi 20 event seperti kondisi laporan.

Studi kasus perbaikan

D1. Tambahkan `$this->call(BukuSeeder::class);` pada method `run()` DatabaseSeeder. Alternatif eksekusi langsung adalah `php artisan db:seed --class=BukuSeeder`, tetapi kartu ini meminta perbaikan kode agar ikut `db:seed`.

D2. Ubah nama file menjadi `resources/views/buku.blade.php`, atau ubah pemanggilan menjadi `view('books')`. Pilih salah satu sesuai nama yang ingin dipakai.

D3. `return view('buku', compact('data_buku'));`

D4. `protected $table = 'books';`

D5. `if (stripos($judul, $keyword) !== false)` — nilai posisi 0 tetap dihitung sebagai ditemukan.

D6. `php artisan config:clear`.

D7. `php artisan migrate`.

D8. `DELETE FROM buku WHERE id = 7;`.

Lima pasangan yang wajib dibedakan

1. `make:seeder` membuat file; `db:seed` menjalankan pengisian data.

2. `make:migration` membuat file; `migrate` menerapkan perubahan struktur.

3. `php file.php` menghasilkan output terminal; `php -S ...` melayani HTTP; `php artisan serve` menjalankan server development Laravel.

4. `view('buku')` memilih template; `compact('data_buku')` mengirim data.

5. `$data_buku->count()` menghitung Collection itu; `Buku::count()` menghitung baris pada tabel melalui model.